

LAB SESSION 4

Lab Name: Software Requirement Specification (SRS)

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Prepare a formal IEEE-standard SRS document for MediScript-E
 - Learn requirement documentation best practices
 - Establish a contract between stakeholders and developers
-

Theory

The SRS document acts as:

- **A Contract:** Agreement between stakeholders and development team
- **A Design Reference:** Blueprint for system architecture and implementation
- **A Testing Baseline:** Foundation for test case design and validation

A good SRS must be **complete**, **unambiguous**, and **verifiable**. For MediScript-E, the SRS ensures all healthcare workflows are properly documented before implementation.

Software Requirement Specification Document

1. Introduction

1.1 Purpose

This SRS document defines the functional and non-functional requirements for **MediScript-E**, a digital healthcare platform. It serves as a reference for developers, testers, and stakeholders throughout the software development lifecycle.

1.2 Scope

MediScript-E is a web-based healthcare platform that enables:

- Patients to book appointments, manage prescriptions, set medicine reminders, and store medical records
- Doctors to manage appointments and issue digital prescriptions
- Administrators to monitor platform activity and manage users

The system is built using **Next.js 16**, **PostgreSQL (Supabase)**, **Prisma ORM**, and deployed on **Vercel**.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirement Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
OTP	One-Time Password
2FA	Two-Factor Authentication
JWT	JSON Web Token
API	Application Programming Interface
PDF	Portable Document Format
OAuth	Open Authorization

1.4 References

- Next.js 16 Documentation: <https://nextjs.org>
- Prisma ORM Documentation: <https://prisma.io>
- Supabase Documentation: <https://supabase.com>
- NextAuth.js Documentation: <https://next-auth.js.org>
- IEEE Std 830-1998: IEEE Recommended Practice for SRS

1.5 Overview

This document is organized as follows:

- Section 2: Overall system description
- Section 3: Functional requirements
- Section 4: Non-functional requirements
- Section 5: Constraints and assumptions

2. Overall Description

2.1 Product Perspective

MediScript-E is a standalone web application that integrates with:

- **Supabase** for PostgreSQL database and file storage
- **Google & GitHub OAuth** for third-party authentication
- **Gmail SMTP (Nodemailer)** for email notifications
- **Vercel** for cloud deployment

2.2 Product Functions

The system provides the following major functions:

- 1. User authentication and authorization (email/password, OAuth, 2FA)
- 2. Appointment booking and management
- 3. Digital prescription issuance and PDF download
- 4. Medicine reminder scheduling with email alerts
- 5. Medical vault for secure document storage
- 6. Admin dashboard with real-time statistics

2.3 User Classes and Characteristics

User Class	Description	Technical Level
Patient	Registers, books appointments, manages health records	Low to Medium
Doctor	Manages appointments, issues prescriptions	Medium
Admin	Monitors platform, manages users	High
Public	Views landing page, submits contact form	Low

2.4 Operating Environment

- **Client:** Modern web browsers (Chrome, Firefox, Safari, Edge)
- **Server:** Vercel Serverless Functions (Node.js runtime)
- **Database:** PostgreSQL hosted on Supabase (AWS ap-south-1)
- **Storage:** Supabase Storage for medical documents

2.5 Design and Implementation Constraints

- Must use Next.js App Router architecture
- Must use Prisma ORM for all database operations
- Must comply with Vercel serverless function limitations
- File uploads limited to supported medical document formats

2.6 Assumptions and Dependencies

- Users have access to a valid email address
- Users have a stable internet connection
- Supabase services remain available
- Gmail SMTP credentials remain valid for email delivery

3. Functional Requirements

3.1 Authentication Module

FR ID	Requirement	Priority
FR-01	System shall allow users to register with name, email, password, and role (PATIENT/DOCTOR)	High
FR-02	System shall send an email verification link upon successful registration	High
FR-03	System shall prevent login until email is verified	High
FR-04	System shall allow verified users to login with email and password	High
FR-05	System shall support OAuth login via Google and GitHub	Medium
FR-06	System shall auto-verify OAuth users and create a patient profile	Medium
FR-07	System shall allow users to enable 2FA email OTP from settings	High
FR-08	System shall send a 6-digit OTP to user email when 2FA is enabled during login	High
FR-09	System shall invalidate OTP after 10 minutes or successful use	High
FR-10	System shall allow users to resend OTP with a 60-second cooldown	Medium

3.2 Patient Module

FR ID	Requirement	Priority
FR-11	System shall display list of available doctors for appointment booking	High
FR-12	System shall allow patients to book appointments with date, time, and reason	High
FR-13	System shall allow patients to view all their appointments with status	High
FR-14	System shall allow patients to cancel pending appointments	Medium
FR-15	System shall allow patients to view prescriptions issued by doctors	High

FR ID	Requirement	Priority
FR-16	System shall allow patients to download prescriptions as PDF	High
FR-17	System shall allow patients to add medicine reminders with name, dosage, frequency, and schedule	Medium
FR-18	System shall send automated email alerts for medicine reminders via cron job	Medium
FR-19	System shall allow patients to mark medicine as taken or undo	Medium
FR-20	System shall allow patients to upload medical documents to Medical Vault	Medium
FR-21	System shall allow patients to delete uploaded medical documents	Low
FR-22	System shall display patient blood group selected during registration	Low

3.3 Doctor Module

FR ID	Requirement	Priority
FR-23	System shall display all appointments assigned to the doctor	High
FR-24	System shall allow doctors to confirm pending appointments	High
FR-25	System shall allow doctors to cancel appointments	High
FR-26	System shall allow doctors to mark appointments as completed	High
FR-27	System shall allow doctors to issue prescriptions with patient ID, diagnosis, and medications	High
FR-28	System shall display doctor specialization and license number on profile	Medium

3.4 Admin Module

FR ID	Requirement	Priority
FR-29	System shall display real-time statistics: total users, patients, doctors, appointments, prescriptions, contacts	High
FR-30	System shall allow admin to view all registered users with their profiles	High
FR-31	System shall allow admin to delete any user except themselves	High
FR-32	System shall allow admin to view all appointments with status filter (ALL/PENDING/CONFIRMED/COMPLETED/CANCELLED)	Medium
FR-33	System shall allow admin to view all contact form submissions	Low

3.5 General Module

FR ID	Requirement	Priority
FR-34	System shall allow users to update their display name	Medium
FR-35	System shall allow users to change their password with current password verification	Medium
FR-36	System shall allow public users to submit a contact form with name, email, phone, and company	Low

4. Non-Functional Requirements

4.1 Performance

- **NFR-01:** Pages shall load within 3 seconds under normal network conditions
- **NFR-02:** API responses shall complete within 2 seconds for standard database queries
- **NFR-03:** PDF generation shall complete within 5 seconds

4.2 Availability

- **NFR-04:** System shall maintain 99.9% uptime in production environment
- **NFR-05:** Scheduled cron jobs for medicine reminders shall execute with less than 5-minute delay

4.3 Security

- **NFR-06:** All passwords shall be hashed using bcryptjs with minimum 10 salt rounds
- **NFR-07:** All database connections shall use SSL/TLS encryption
- **NFR-08:** JWT sessions shall expire after 30 days of inactivity

- **NFR-09:** Email verification tokens shall expire within 24 hours
- **NFR-10:** 2FA OTP codes shall expire within 10 minutes
- **NFR-11:** All API routes shall implement role-based access control (RBAC)
- **NFR-12:** Medical file uploads shall be validated for file type and size
- **NFR-13:** All user inputs shall be validated and sanitized before processing

4.4 Usability

- **NFR-14:** System shall be fully responsive across mobile, tablet, and desktop devices
- **NFR-15:** System shall provide clear success and error feedback for all user actions
- **NFR-16:** System shall support smooth page transitions and animations

4.5 Maintainability

- **NFR-17:** System codebase shall follow TypeScript strict typing standards
- **NFR-18:** All database operations shall use Prisma ORM for consistency
- **NFR-19:** System shall follow Next.js App Router conventions

4.6 Scalability

- **NFR-20:** System shall be deployable on Vercel with zero-downtime deployments
- **NFR-21:** Database connection pooling shall be configured for concurrent users

5. Constraints & Assumptions

5.1 Constraints

- System must be deployed on Vercel platform
- Database must use PostgreSQL via Supabase
- Email notifications require valid Gmail SMTP credentials
- OAuth login requires valid Google and GitHub OAuth application credentials
- Cron job for medicine reminders requires a valid CRON_API_KEY
- File storage requires a configured Supabase storage bucket named **medical-reports**

5.2 Assumptions

- All users have access to a valid and active email address
- Doctors provide valid medical license numbers during registration
- Admin account is pre-seeded in the database
- Users have access to modern web browsers with JavaScript enabled
- Internet connectivity is available for all users during system use

Key Findings / Learning Outcomes

- Learned to prepare a formal IEEE-standard SRS document for a real-world healthcare application
- Understood how SRS acts as a contract between stakeholders and development team

- Identified **36 Functional Requirements** across 5 modules and **21 Non-Functional Requirements** across 6 categories
- Understood the importance of constraints and assumptions in requirement documentation
- Recognized that a well-structured SRS reduces ambiguity and serves as a baseline for testing